

From Vertex AI to Edge-Ready LLMs

A Hybrid Cloud-Edge Framework for
Privacy-Preserving Healthcare Inference

Anton J. Rasmussen

Department of Computer Science
Old Dominion University

CS795 / CS895 – Generative AI
Instructor: Dr. Hong Qin
Fall 2025

Project Motivation

The Problem

- Large Language Models are **powerful but expensive**
- Most healthcare data is **sensitive and regulated**
- Cloud-only inference:
 - Introduces privacy risk
 - Adds latency
 - Limits personalization

Core Question

Can we combine cloud-scale training with efficient, local inference for healthcare AI?

Vision

Big Picture

- **Train & orchestrate** models in the cloud (Vertex AI)
- **Compress & deploy** models locally (CPU / edge devices)
- **Keep patient data local**
- Enable future **federated or collaborative learning**

This project explores the **foundations** needed to make that possible.



Abstract

This project investigates a hybrid cloud-edge framework for deploying Large Language Models in privacy-sensitive healthcare settings. Using Google Vertex AI as a cloud orchestration layer, I explore post-training model compression techniques and local inference strategies aimed at enabling efficient, CPU-based deployment of LLMs.

Through a sequence of experiments—including Vertex AI model deployment, dynamic quantization, and a federated learning prototype—this work evaluates the feasibility, limitations, and future potential of edge-ready LLMs for personalized healthcare applications.



Project Structure

Three Phases

1. Phase I – Foundations

- Vertex AI deployment & MLOps workflow

2. Phase II – Compression

- Quantization experiments for edge inference

3. Phase III – Collaboration (Future)

- Federated learning & decentralized intelligence

This course project focuses primarily on **Phases I and II**.



Phase I: Vertex AI Foundations

Goal

Establish an **end-to-end cloud workflow** for training, registering, and deploying models.

Why Vertex AI?

- Managed MLOps
- Model registry & versioning
- Batch + online inference
- Scales to future LLM workloads

Vertex AI Workflow

Implementation Steps

1. Train a `RandomForestClassifier` on the Iris dataset
2. Serialize model artifacts
3. Upload to Vertex AI Model Registry
4. Deploy to a live endpoint
5. Execute online & batch predictions

Key Takeaway

This validated the **cloud-side foundation** needed for later LLM experimentation.



Lessons from Phase I

Challenges Encountered

- Serialization mismatches
- Python / scikit-learn version conflicts
- Limited access to Cloud Logging
- IAM permission constraints

What This Taught Me

- Real-world MLOps is fragile
- Reproducibility matters
- Version alignment is critical



Phase II: Why Compression Matters

Problem

LLMs are:

- Too large for CPUs
- Too slow for embedded systems
- Too expensive for continuous cloud inference

Solution Space

- Post-training quantization
- Mixed precision
- Model pruning
- Efficient runtimes (e.g., `llama.cpp`)



Compression Strategy

Initial Focus

- **Post-training quantization**
- No retraining required
- Lower barrier to experimentation

Techniques Explored

- PyTorch Dynamic INT8 Quantization
- GPTQ / AWQ (partially supported)
- GGUF export pipeline (planned)



Quantization Experiment

Setup

- Model: `distilgpt2`
- Environment: CPU-only (Colab)
- Metric: Latency, size, compression ratio

Method

- Baseline FP32 inference
- Apply dynamic INT8 quantization
- Compare performance



Quantization Results

Model	Latency (32 tokens)	Size (MB)	Speedup	Compression
FP32	1.001 s	312.5	1.00×	1.00×
INT8	1.188 s	349.3	0.84×	0.89×

Key Observation

Dynamic quantization **did not help** for small models.



Interpretation

What This Means

- Small models may not benefit from naive quantization
- Overhead can outweigh gains
- Advanced methods are required for real savings

Why This Result Still Matters

- Establishes a **baseline**
- Validates experimental pipeline
- Guides future optimization choices



Federated Learning Prototype

Motivation

Healthcare data is:

- Distributed
- Sensitive
- Often cannot be centralized

Idea

Use **federated averaging (FedAvg)** to combine model updates without sharing raw data.



FedAvg Simulation

Experiment

- Synthetic client updates
- Simple vector averaging
- Validates aggregation logic

Result

```
Global update:  
[1.05, 2.05, 3.05]
```



Why FedAvg Matters Here

- Enables collaborative learning
- Preserves privacy
- Fits edge deployment
- Supports future personalization



Demo (Optional / Recorded)

Demonstration Options

- Vertex AI endpoint inference
- Quantization script walkthrough
- CPU-only inference benchmark
- FedAvg aggregation code



Discussion

What Worked

- Vertex AI deployment pipeline
- Modular experimentation
- CPU-only workflows
- Reproducibility

What Didn't (Yet)

- Stable Phi-3 quantization
- AWQ / GPTQ compatibility
- Meaningful speedups on small models



Future Work

Technical

- Full GPTQ / AWQ on Phi-3 or Mistral
- GGUF + `llama.cpp`
- Edge benchmarking
- Vertex AI custom jobs

Research

- Federated fine-tuning
- Edge-RAG
- Personalized health inference



Healthcare Impact

- Keeps patient data local
- Reduces cloud dependence
- Enables personalization
- Improves trust & transparency



Acknowledgments

- Dr. Hong Qin
- CS795 / CS895 classmates
- Open-source ML community
- Google Vertex AI resources

? Q & A

Thank you – I welcome questions and feedback.